BOOTCAMP III / ARQUITETURA DE SOFTWARE

Documento de Design do Sistema

TaskAnalyzer | Especificação de arquitetura alinhada ao contrato SDD da Fase 1 (13/09/2026, v1.0).

01 Objetivo e escopo

O módulo calcula indicadores de produtividade de tarefas a partir de registros em memória. A interface principal preserva os nomes, unidades e restrições da Entrega Inicial. Uma interface suplementar atende aos totais e à unidade em horas solicitados na Fase 2, sem modificar o contrato original.

02 Visão de componentes

Componente	Responsabilidade
Entrada JSON / interface Python	Receber tarefas com id numérico, datas UTC, prazo, prioridade e status.
Validação de contrato	Validar campos, valores, fuso e ordem cronológica. Emitir TaskValidationError em erro.
Agregação SDD	Somar durações em minutos apenas de concluídas; contar atrasos concluídos; agrupar por prioridade.
Resumo Fase 2	Adapter separado: contagens por status e conversão da média para horas.
Test Harness / CI / Docker	Automatizar cenários de sucesso, exceção e borda nos ambientes local e isolado.

03 Fluxo de dados

Lista de tarefas → validação de campos e UTC → registros imutáveis → agregação geral e por prioridade → arredondamento → JSON de saída.

O módulo não utiliza banco de dados nem grava os registros recebidos. Os arquivos em examples/ são entradas fictícias de demonstração, lidas sem atualização.

04 Regras de cálculo

Métrica	Definição implementada
Duração	(data_conclusao − data_inicio) / 60 segundos: duração em minutos.
Média	Soma das durações / quantidade de concluídas; 0.0 quando não houver concluídas.
Atraso	Uma concluída atrasa se data_conclusao > prazo. Pendente/cancelada não é atraso.

Métrica	Definição implementada
Taxa de atraso	Concluídas atrasadas / concluídas × 100; 0.0 sem concluídas.
Quantidade	Todos os registros válidos; inclui concluídas, pendentes e canceladas.

Interpretações não expressas integralmente na Fase 1 (denominador da taxa e contagem por status) foram documentadas, e não apresentadas como texto literal da especificação.

05 Política de validação e resolução de divergências

A Fase 1 apresenta uma regra para ignorar datas inconsistentes, mas seu Cenário 2 exige exceção específica; a Fase 2 também solicita TaskValidationError para conclusões anteriores à criação. O projeto adota rejeição explícita de entradas inválidas, evitando cálculos parciais sem conhecimento do usuário.

A Fase 1 determina retorno com quatro campos. A Fase 2 exige indicadores complementares. O projeto mantém analyze_tasks com quatro campos e fornece resumo_fase2 como adaptador. Essa separação evita mudanças silenciosas da API.

06 Critérios de qualidade e restrições

Python 3.11+, type hints, funções pequenas, responsabilidades separadas, logs estruturados com logging, exceções específicas, ausência de persistência e dependência externa somente para testes (pytest). O projeto usa UTC explícito e arredondamento em duas casas decimais.

07 Mapa de arquivos

Caminho	Função
src/task_analyzer.py	Validação, cálculos, APIs e interface de linha de comando.
tests/test_harness.py	Testes de aceite, borda, validação, CLI e invariantes.
specs/task_analyzer_spec.md	Especificação de entradas, saídas, decisões e exemplos.
CONTEXT_RULES.md	Regras para IA e homologação humana.
.github/workflows/ci.yml	Jobs automatizados pytest e Docker Compose.
Dockerfile / docker-compose.yml	Imagem e execução reproduzível da suíte.
docs/relatorio_governanca_etica.pdf	Análise crítica de IA, conformidade e evidências.

08 Limites e evidências

A suíte de testes teve 65 testes aprovados localmente durante a preparação do projeto. A execução real no GitHub Actions e no Docker local não foi comprovada nesta elaboração e deverá ser validada após a publicação.